

Available online at www.sciencedirect.com

ScienceDirect

journal homepage: www.keaipublishing.com/foar

RESEARCH ARTICLE

Generating floor plan diagrams using a self-organising adjacency graph

Mohamed Zaghloul ^{a,b,*}, Ludger Hovestadt ^c

^a Healthy and Sustainable Built Environment Research Center (HSBERC), College of Architecture, Art and Design (CAAD), Ajman University, Ajman, United Arab Emirates

^b Artificial Intelligence Research Center (AIRC), Ajman University, Ajman, United Arab Emirates

^c Chair of Digital Architectonics, ETH Zürich, Zürich 8093, Switzerland

Received 21 May 2025; received in revised form 4 November 2025; accepted 24 November 2025

KEYWORDS

Floor plan diagram;
Machine learning;
Self-organising map;
Unsupervised learning;
Graph distance function;
Cosine distance function

Abstract In this study, we introduce a novel self-organising map algorithm-based approach to floor plan diagram generation. A graph distance function was used instead of the standard Euclidean distance, together with a graph grid that fits inside any floor plan shape. The cosine distance was used to determine the best-matching neuron in the training process. Machine learning was then used to optimise the diagram on the fly; specifically, an unsupervised learning technique automatically generated initial floor plan diagrams without the need for an existing training dataset. The proposed method uses an adjacency matrix that describes the relationship between spaces. As a vital part of the process, the user can initialise fixed positions for some zones and elements, such as the entrance, thereby influencing the adjacency zoning localisation outcome. A Mathematica interface allows users to set input entrance positions, draw floor plan outlines and specify the direct relations between spaces. To validate the feasibility of the proposed approach, we performed an experiment using a residential apartment. The main contribution of this study was in improving the automation of the initial planning and design stages, which are the first crucial stages in designing floor plans.

© 2026 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

* Corresponding author. Healthy and Sustainable Built Environment Research Center (HSBERC), College of Architecture, Art and Design (CAAD), Ajman University, Ajman, United Arab Emirates.

E-mail address: m.zaghloul@ajman.ac.ae (M. Zaghloul).

Peer review under the responsibility of Southeast University.

<https://doi.org/10.1016/j.foar.2025.11.011>

2095-2635/© 2026 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Please cite this article as: M. Zaghloul and L. Hovestadt, Generating floor plan diagrams using a self-organising adjacency graph, Frontiers of Architectural Research, <https://doi.org/10.1016/j.foar.2025.11.011>

1. Introduction

Floor plan diagrams are crucial in the initial stages of the architectural design process. They play a key role in transforming the relationship requirements and spatial considerations of a project into tangible layouts. Traditional methods of drawing an architectural diagram of a floor plan focus on achieving connectivity between spatial zones using adjacent shapes as a group of zones. In 1864, architect Robert Kerr proposed what he termed a “thoroughfare plan.” He described a similar abstract conceptual sketch as a “skeleton upon which rooms are grouped,” prioritising movement and functional relationships over “spatial detail.” At that time, he drew only adjacent rectangles to denote groups of rooms (Emmons, 1998). Bubble diagram sketches, consisting of circles and connecting lines, are used to indicate relationships between spaces and functions; they have long been used as a form of abstraction in architectural programming, especially in modern architecture. In his 1938 essay “If I Had to Teach You Architecture”, Le Corbusier described a similar method, starting with a straight line and building outward with functional units, thereby emphasising clarity and organisation (Fig. 1, left diagram as an abstract visual interpretation). In this approach, the diagram was more than just a technical tool; it was considered a generator of architectural logic from the inside out (Emmons, 1998).

In 1937, Nobbs used the idea of a network diagram (Fig. 1, middle diagram as an abstract visual interpretation) followed by the steps of assembling elements. Nobbs conceived the design process of building as the evolution of an organism and suggested using the diagram as a tool for simulating an optimal solution. He explains the bubble diagram as follows: “the plan at this stage is like a medical student’s dissected animal, with all its organs neatly spilled out and some of the connections stretched, but none broken” (Nobbs, 1937). However, this approach involves manually drawing and making iterative adjustments, which can be time-consuming. It is also challenging to efficiently capture the complex spatial relationships inherent to

architectural design. During the 20th Century, the representation of bubble diagrams (Fig. 1, right diagram as an abstract visual interpretation) (White, 1975) became a foundational design method and a guiding design framework for architects to uncover and systematise space-planning procedures (Hsu and Krawczyk, 2003).

In 1957, the first computer algorithms were used to solve “spatial allocation problems.” Alexander (1964) framed the design process as a mathematical and algorithmic process transforming requirements into a graph-like network with a hierarchical, clustered network of nodes, revealing the relationships between spaces (Fig. 2, left, as an abstract visual interpretation). Between 1957 and 1970, more than thirty different programs were developed to help automate stages of the generation of building layouts (Shaviv, 1985). Then, between the 1970s and 2000, four main approaches for generating a bubble diagram by computer algorithms that are typically transformed into a layout plan were applied. These are as follows:

- a) Rule-based space planning (Fig. 3, as an abstract visual interpretation); initially proposed by Mitchell and Dillon (1972). This includes form-generation rules through which computational procedures can rapidly produce a wide range of floor plan diagrams.
- b) Physics-based bubble diagrams, introduced by Fortin (1978). This starts with a random position of spaces; then elements are placed according to attractive and repulsive forces related to adjacency rules. This method relies on the initial condition and is difficult to balance in the case of a complex project relationship (Fortin, 1978).
- c) Graph theory-based bubble diagram, proposed by Ruch (1978) as an interactive programming for the graph, bubble diagram and schematic plan (i.e., Fig. 2, middle and right, as an abstract visual interpretation). During the 1980s, it was proven that a bubble-diagram graph could be captured and manipulated on screen; later, in the early 1990s, its generation was automated using constraint-based engines; then, in the late 1990s,

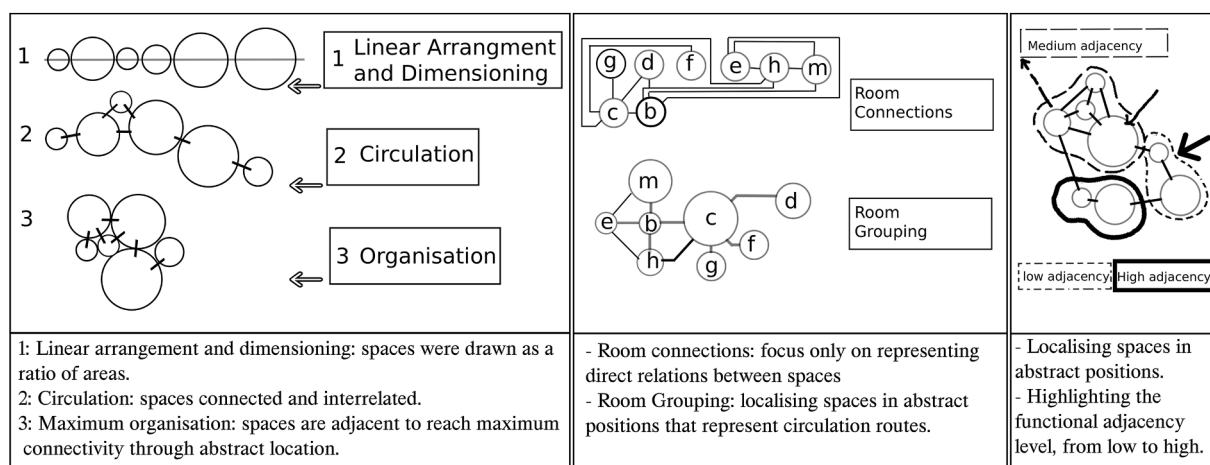


Fig. 1 Left: a conceptual bubble diagram inspired by Le Corbusier based on the description of Lueder (2012). Middle: a conceptual room connections diagram inspired by Percy Nobbs’ 1932 based on the description of Nobbs (1937). Right, a conceptual bubble diagram inspired by Edward White’s 1975 based on the description of White (1975).

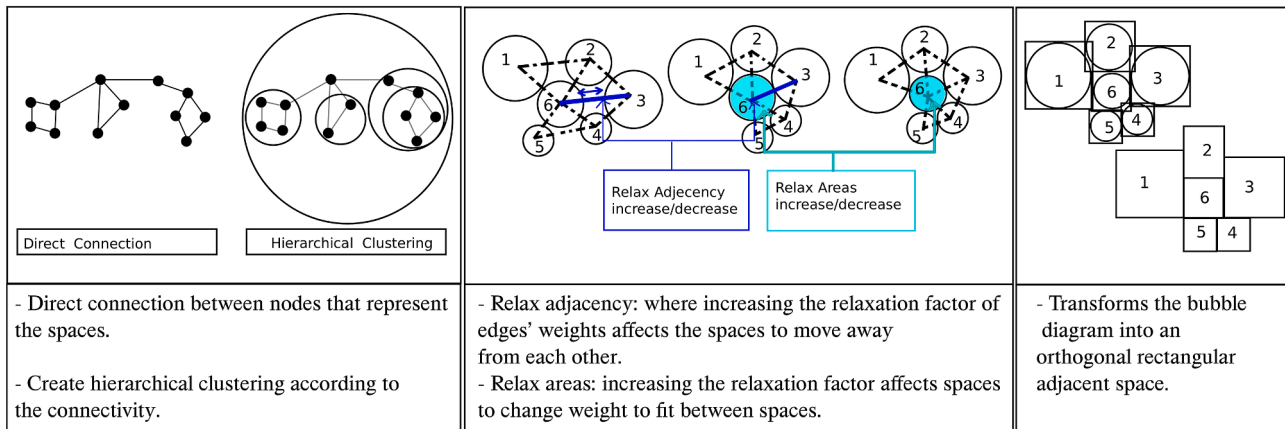


Fig. 2 Left: a conceptual graph of a hierarchical clustered network inspired by the model described by [Alexander \(1964\)](#). From the middle conceptual graph of automating relaxes areas through iterative steps, to the right conceptual graph of the transformation of the bubble diagram into an orthogonal grid plan, inspired by the model described by [Ruch \(1978\)](#).

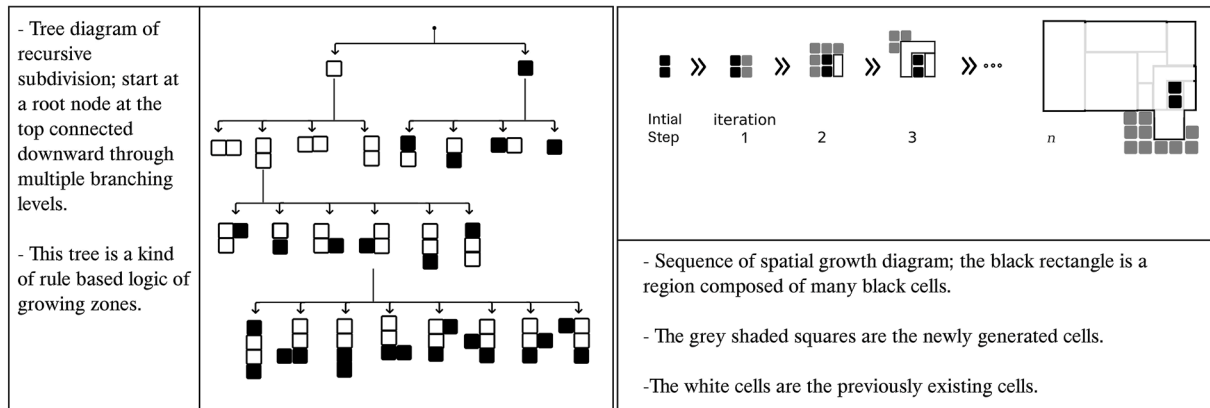


Fig. 3 Left: a conceptual tree diagram of the rule-based logic of growing zones. Right: a conceptual spatial growth diagram inspired by based on the model described by [Mitchell and Dillon \(1972\)](#).

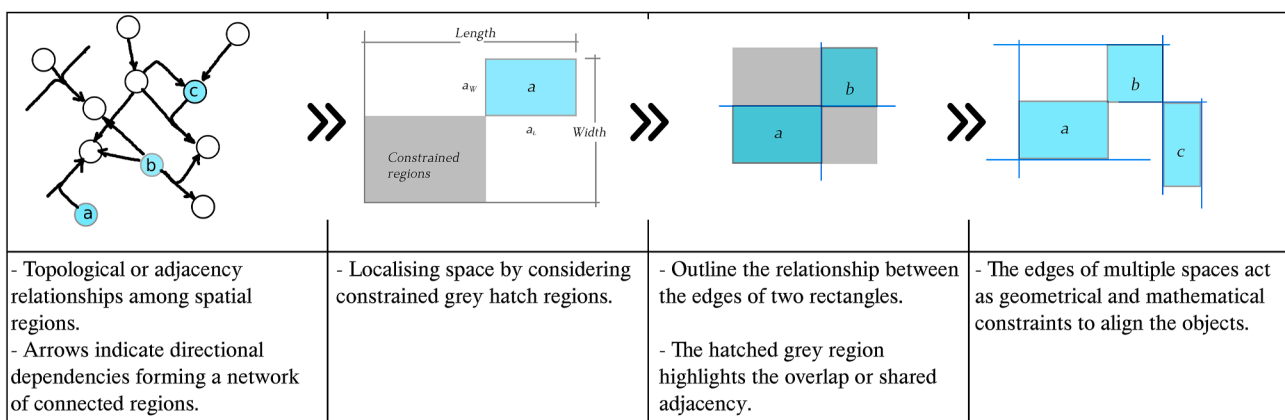


Fig. 4 Left: a conceptual diagram starts from a topological constraint graph; then selecting zones involves successive geometrical steps to define areas and adjacency of zones, inspired by the model described by [Galle \(1981\)](#).

evolutionary searches were used to evolve whole families of graphs, treating the bubble diagram itself as a design variable rather than a static sketch ([Hsu and Krawczyk, 2003](#)).

d) Mathematical-based programming, which typically uses graph theory as a topological constraint graph, then defines geometrical mathematical constraints of rooms ([Fig. 4](#), as an abstract visual interpretation of the

successive steps); such as areas and positions (e.g., Galle, 1981).

Since 2000, these four approaches have continued to be used, but with different algorithms. A new fifth approach has utilised artificial neural networks (ANN) to develop a comprehensive layout based on images, leveraging databases that are limited to specific typologies (residential, commercial, industrial) for each database. Recent related studies are presented in detail in the following section. We highlight that our study proposes a new approach for using ANN to remap the adjacency relations amongst architectural elements. Unlike current ANN approaches, it does not rely on a specific database, nor does it seek to automate the entirety of the process while generating a final detailed floorplan with walls. Rather, it provides the user with an interactive canvas in which automated mapping of topological relationships can be applied to initial geometrical bubble diagrams.

2. Related research

Several recent studies have explored computer algorithmic processes for generating floor plan diagrams efficiently while considering diverse design requirements, all of which have developed algorithms similar to the four previous methodological approaches:

- a) Rule-based space planning. Guo and Li (2017) developed an evolutionary agent-based approach that uses physical interaction rules (attraction, swap, repulsion, compression) and is well-suited to the conceptual design of spaces with no complex geometric conditions, such as irregular plots.
- b) Physics-based bubble diagrams. Attraction and repulsion forces are the main forces required to solve the adjacency of connected bubbles and pack them inside a user-defined polygon within a 3D modeling tool, such as in Veloso et al. (2014).
- c) Graph-theory-based bubble diagram. This uses graphs to organise zones and spatial relationships. Various representations, including visibility graphs, Voronoi diagrams and adjacency matrices have been employed to capture connectivity and adjacency constraints (Franz et al., 2005; Nassar, 2019). These approaches support designers in conceiving spatial configurations but often require extensive manual modification. Most such approaches use physics-based spring computations. Other researchers have developed graph-based algorithms for generating only rectangular floor plans, allowing the insertion of new rooms while maintaining the connectivity of the existing layout (Wang et al., 2018). The standard method used in this approach is the spring embedder graph-drawing algorithm, which serves as the basis for many modern models. This algorithm solves an optimisation problem in which the energy of a configuration is minimised while ensuring that every vertex of the graph remains within specified boundaries and maintains the characteristics inherent to force-directed methods in the final layout (Zhang and Pang, 2016).
- d) Mathematical-based programming (parametric method); other studies have used parametric modelling tools to enable designers to participate actively in the generation process by specifying design constraints based on graphs and preferences (Egor et Figal., 2020; Hu et al., 2020). These techniques facilitate the iterative exploration of design alternatives, relying on physics-based graph optimisation within a parametric manipulation; however, they do not enable real-time exploration of a context or outline shape.
- e) Machine learning techniques have also been used to automate various aspects of architectural design, including floor plan generation. Generative adversarial networks and convolutional neural networks have been used to learn patterns from existing floor plans and generate new layouts (Nauata et al., 2021; Yıldız et al., 2023). Although effective, these methods rely on large training datasets and may fail to capture complex spatial relationships, especially when the designer applies a new context or project requirement that is absent from the dataset. The primary goal of using these techniques is to obtain a detailed layout with walls and doors. Here, it is necessary to highlight that even this study uses a machine learning method; however, it is used in a manner such that there is no reliance on a specific dataset. Nonetheless, it focuses on producing a bubble diagram where the nodes are localised in space with respect to relationships and areas.

Current approaches generally minimise human participation, limiting the potential for deeper integration of AI into design practice, and their focus is on finalising the layout with details. In addition, the deep learning techniques used in most of the studies (Veloso et al., 2022; Wu et al., 2019; Zhang et al., 2024) were unable to generalise strategies to unseen scenarios, preventing the use of the developed approaches as a scalable design method for architectural design. Using the reinforcement learning approach, Kakooee et al. (2024) and Kakooee and Dillenburger (2025) observed that an agent trained on a particular design scenario often struggles to produce satisfactory solutions when confronted with new design requirements or constraints, necessitating retraining for each scenario. Their study, like most preceding efforts, tried to automate all of the layout phases, from topological fitting to the final detailed wall and geometrical fitting.

In contrast, our study focuses on using ANN without data, and the neurons are a probabilistic space in which the designer explores positioning zones and areas according to the probability of real-time interactions with the competitive learning process between the neurons. In this study, we introduce a new method for creating floor plan diagrams, using a self-organising map (SOM) algorithm enhanced with a graph distance function explicitly tailored for architectural applications. SOMs have been widely applied in data visualisation, clustering, abstraction and dimensionality reduction. They consist of processing elements called neurons or nodes, which are interconnected with associated weights. By organising input data onto a lower-dimensional grid while preserving topological relationships, SOMs offer a powerful tool for analysing, understanding and reducing high-dimensional data (Kohonen, 2001). They also reflect the topological structure of data, making them invaluable for interpreting complex graph relationships (Bonabeau and

Hénaux, 1998). SOMs have also been explored in architectural applications for generating in-between architectural spaces for previous designs (Zaghloul, 2015).

However, their use in floor plan generation has been limited to design generation based on complete design data, rather than abstract relations between spaces. The inverted SOM is an algorithm developed for the rapid generation of complex graphs (Meyer, 1998) in which the user does not interact with the training process to fix graph node positions; rather, it uses a generic distance metric. Unlike conventional SOM methods, which predominantly use Euclidean distance metrics, the proposed approach uses the cosine distance function, representing the angular relationship between two vectors. The cosine distance values range between 0 for identical vectors and 2 for opposite vectors. A value of 1 denotes that the vectors are orthogonal. This approach leverages the inherent topology of spatial relationships encoded in graph structures, allowing for more accurate and contextually relevant floor plan generation.

Central to our methodology is the use of SOM neurons as topological and metric positions of probabilistic zones. As an unsupervised learning technique, SOMs can map high-dimensional input vectors into a lower-dimensional grid while preserving topological relationships. Building on the SOM approach, we seek to empower users to influence floor plan generation processes by setting fixed positions for key elements, such as entrances and viewpoints. This interactive feature enables designers to integrate their intent into the generated floor plans in real time, resulting in contextually relevant outcomes. Various experiments were conducted on floor plan diagrams of residential buildings to validate our method and demonstrate its capability to generate initial layouts in real time. The results indicate the potential of the self-organising adjacency graph (SOAG) method to initiate floor plan diagrams, offering architects and designers an efficient and intuitive tool with which to explore design alternatives while meeting project requirements.

Informed by the limitations of these diverse research efforts, our proposed SOAG method advances the automated generation of floor plan diagrams by adapting the standard SOM algorithm. First, the SOM neurons are represented as a meaningful geometric grid inside the floor plan outline function while projecting the adjacency matrix. Second, the cosine distance is used to select the winner neurons, which are termed the best matching unit (BMU). Lastly, specific zone elements are localised next to some neurons, which forces the SOAG to work as a diagram optimiser. This integration automatically generates initial floor plan diagrams while accommodating user input and spatial constraints. These optimisation objectives are integrated at the front-end level, where the user interacts with the training process in real time.

3. Methodology

The SOAG algorithm, developed for architects and using Mathematica as its background coding language, is packaged as a Wolfram Player file. In the first step, users define the geometry of the floor plan by importing an outline from any CAD tool or by drawing it inside a basic canvas. Then,

the topological requirements are defined by specifying the spaces that have a direct relationship in a simple data sheet input. Next, the user suggests the positions of key points, such as the entrance and outer viewpoints, and can enter the area of each space. In this study, the terms zone and space are used interchangeably to describe broader areas within the bubble diagram, while key points denote the importance of attractive regions. This is effective because the focus is limited to bubble diagrams without detailed walls, doors or other architectural elements; instead, these distinctions remain at a conceptual rather than constructional level. Finally, the SOAG algorithm optimises the position of each element of the floor plan to match the defined topological inputs.

The proposed algorithm focuses on feature engineering the topological relationships and training a SOAG to fit the relational information into a floor plan shape. In this section, we outline the step-by-step process, which is also illustrated in Fig. 5.

3.1. Floor plan graph initiation

The initial step involves creating an outline of the floor plan representation and converting its internal space into a graph grid such that the grid spacing is similar to that of the design metric unit. A polyline P represents the outline of a floor plan, defining its boundaries and layout. The polyline is filled with a grid of graph nodes. Thus, the space inside the floor plan is discretised into a uniform grid of nodes, $G = \{v_1, v_2, \dots, v_j\}$, each representing a specific point in the layout. This graph provides the foundation for representing spatial relationships and facilitates feature extraction for SOAG training.

3.2. Feature engineering

Key points, $K = \{k_1, k_2, \dots, k_m\}$, such as the position of the main entrance, outer viewpoints, or contextual architectural features, are identified, assigned feature vectors, and added to the project zone requirements. Abstract elements, such as viewpoints, are represented as zones with zero area. The relationships between rooms and key points are determined using an adjacency matrix, A , which is derived from the connectivity of rooms and the spatial requirements of the project. For each key point, k_i , a feature vector, $f_{k_i} \in \mathbb{R}^L$, is computed, incorporating spatial, relational and semantic information from A . The feature vector, f_{k_i} , is assigned to the closest graph node, v_{j^*} , based on the Euclidean distance.

3.3. Weight initialisation

The weights of all graph nodes are initialised as follows: first, nodes corresponding to key points are assigned their respective feature vectors, then all other nodes are initialised according to the initial key point vectors, serving as placeholders to be updated during SOAG training. Thus, in this study, only the nodes next to the key points have fixed feature vectors. As a result, the training will not be biased towards a random weight initialisation. Many studies (e.g., Akinduko et al., 2016; Su et al., 1999) have demonstrated

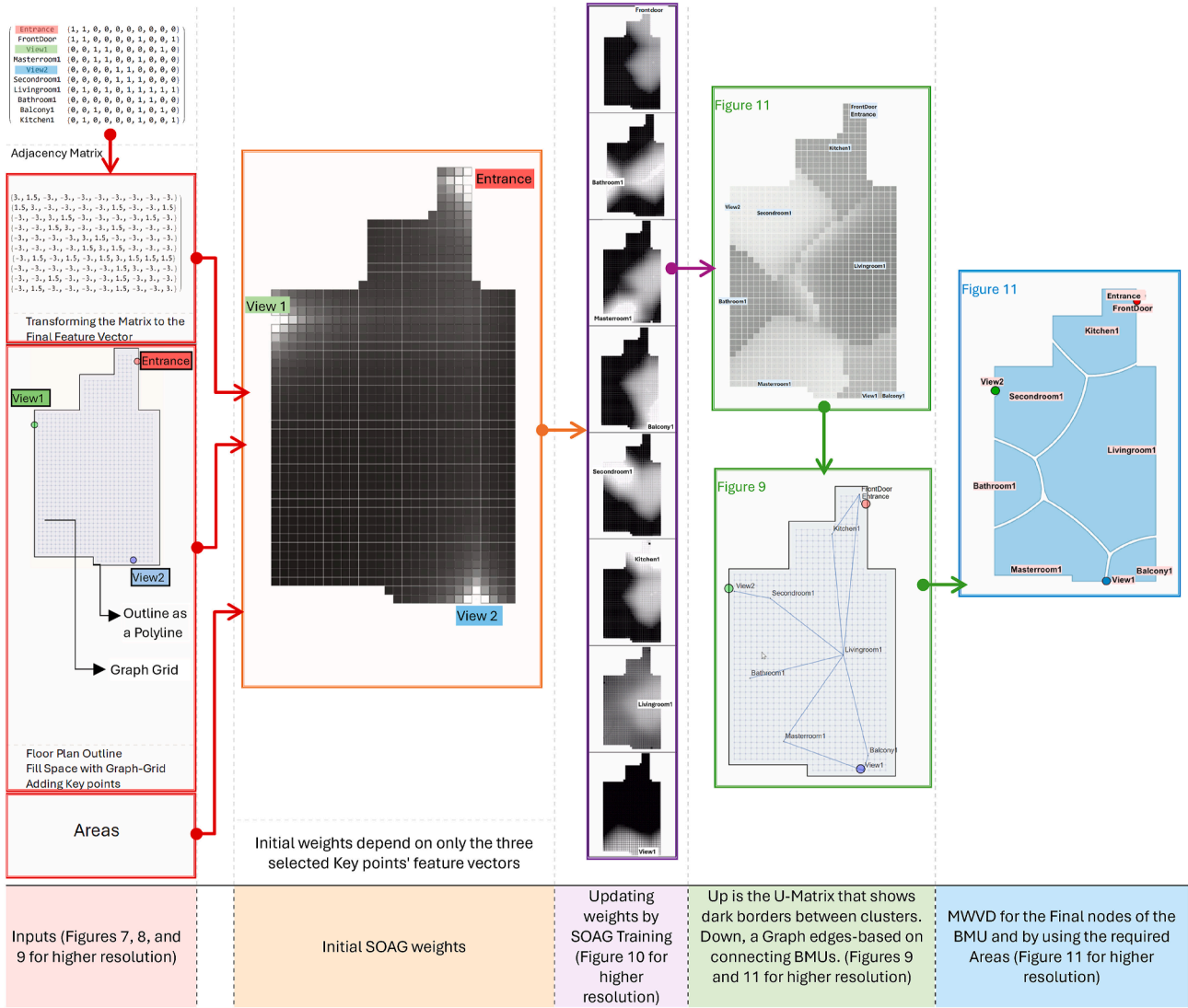


Fig. 5 The SOAG workflow.

that the initialisation of SOM weights plays a crucial role in shaping convergence speed, stability, and overall training quality, making it a key factor to consider in both initial training and retraining processes.

3.4. SOAG training algorithm

The SOAG algorithm (Fig. 6) clusters and organises the feature vectors, providing a topological representation of the spatial relationships within the floor plan.

Input

P : Floor plan outline (polyline);
 A : Adjacency matrix;
 $K = \{k_1, k_2, \dots, k_m\}$: Key points;
 L : Length of feature vectors;
 T : Maximum number of SOAG training iterations;
 ε : Quantisation error threshold;
 r : Initial SOAG radius, equal to the maximum graph distance;

α : Learning rate for SOAG;
 w : SOAG weights.

3.5. Output and SOAG interpretation

The final output consists of the trained SOAG weights, which encode the spatial and relational representations of the floor plan. These weights provide insights into the layout structure, key point clustering and semantic relationships, facilitating further analysis and/or visualisation.

4. Experiment

4.1. Application interface—side input panel

Users can select from different methods for calculating boundary shapes, such as “minimum convex polygon,” which creates a convex-hull bounding shape around the multiplicatively weighted Voronoi diagram (MWVD) region,

Input: Floor plan outline polyline $\mathcal{P}$, key points $K = \{k_1, \dots, k_l\}$, initial feature vectors $f(k_i)$, maximum iteration T , quantization threshold ε
Output: Trained weights w , winner neuron as a center of Zone Best Matching Unit (BMU), graph edges E

```

Begin
  // Floor Plan Graph Initialization
1:  Generate a polyline  $\mathcal{P}$  representing the floor plan outline.
2:  Define graph grid  $G = \{v_1, \dots, v_j\}$ , while  $v_j$  are graph nodes populated inside the  $\mathcal{P}$ 
3:  Compute the adjacency matrix  $A$  between the elements of zones according to the user input for direct relations between zones.
  //Graph-based Feature Engineering
4:  Set diagonal elements' values of  $A_{ii} \leftarrow 3.0, \forall i \in \{1, 2, \dots, n\}$ 
5:  Transform matrix  $A$  into  $A'$  by substituting values using:

$$A'_{ij} = \begin{cases} -3.0, & A_{ij} = 0, \\ 1.5, & A_{ij} = 1. \end{cases}$$

6:  For each key point  $k_i \in K$ :
7:      Compute graph node distance  $j^* = \arg \min ||pos(k_i) - pos(v_j)||^2$ 
8:      Assign  $w_j^* \leftarrow f_{k_i}$ 
9:  Complete weight initialization for  $v_j \in G$ :

$$w_j = f_{k_i} \text{ if } j = j^*, \text{ otherwise } = 0.0$$

  //SOAG Training as a batch learning mode is used
10: Assign QuantizationError  $\leftarrow \varepsilon$ ,  $t \leftarrow 0$ 
11: While  $t < T$  and QuantizationError  $> \varepsilon$ :
12:     Decay neighborhood radius:  $\sigma(t) = (1 - \frac{t}{T+1})r$ 
13:     For each  $f(k_i) \in K$ :
14:         Find BMU:

$$BMU = \arg \min d_{cos}(w_i, f_k)$$


$$\text{where } d_{cos}(u, v) = 1 - \frac{u \cdot v}{||u|| \cdot ||v||}$$

15:         Update weights:

$$w_j(t+1) = \frac{\sum_{k=1}^m \theta_{\sigma(t)}(v_j, BMU(k)) \cdot f_{k_i}}{\sum_{k=1}^m \theta_{\sigma(t)}(v_j, BMU(k))}$$


$$\text{where } \theta_{\sigma(t)}(v_j, BMU(k)) = \exp(-\frac{d_{graph}(v_j, BMU(k))^2}{2 \sigma(t)^2}) \cdot \sqrt{area_j}$$

16:     End For
17:     Compute Quantization error:

$$\text{QuantizationError} = \frac{1}{n} \sum_{i=1}^n d_{cos}(w_i, f_k)$$

18:     Update Key points features:  $w_j^* \leftarrow f_{k_i}$ 
     If QuantizationError  $< \varepsilon$  then
20:         End While, else
21:         End If
22: Select final BMUs:  $\arg \min d_{cos}(w_i, f_k)$ 
23: Draw edges  $E$  between BMUs according to the Transform matrix  $A$ 
Return: Trained weight vectors  $w$  and graph connectivity based on the final BMU

```

Fig. 6 The SOAG algorithm.

as mentioned in Section 5. Other default-bounding regions include “minimum rectangle” and “minimum disk” around the cluster nodes. An iterations slider can be used to adjust

the number of SOAG iterations. Interactive zoom and pan sliders help users navigate and adjust the plot view. The “Run” button executes the SOAG training process.

4.1.1. Application interface—graph zones and relationships

The interface defines and manages the spatial zone requirements of the key points, such as entrances and view-points. The SOAG is constrained to preserve the locations of the key points and associates them with the nearest neuron nodes using its feature vector (Fig. 10). Users can specify relationships between zones or key points, such as adjacency, forming a basis for computing the adjacency matrix, i.e., the core of the feature vector (Figs. 8 and 9). Editable fields assign numerical areas to zones, acting as a ratio between the spaces that affect the weight of the Voronoi distance.

4.1.2. Application interface—graph representation

The right panel visualises the spatial layout overlaid with a grid. Coloured regions denote different zones, nodes represent key locations (e.g., "Entrance" and "View-points"), with edges denoting their defined relationships or connections.

Entrance	{1, 1, 0, 0, 0, 0, 0, 0, 0, 0}
FrontDoor	{1, 1, 0, 0, 0, 0, 1, 0, 0, 1}
View1	{0, 0, 1, 1, 0, 0, 0, 0, 1, 0}
Masterroom1	{0, 0, 1, 1, 0, 0, 1, 0, 0, 0}
View2	{0, 0, 0, 0, 1, 1, 0, 0, 0, 0}
Secondroom1	{0, 0, 0, 0, 1, 1, 1, 0, 0, 0}
Livingroom1	{0, 1, 0, 1, 0, 1, 1, 1, 1, 1}
Bathroom1	{0, 0, 0, 0, 0, 0, 1, 1, 0, 0}
Balcony1	{0, 0, 1, 0, 0, 0, 1, 0, 1, 0}
Kitchen1	{0, 1, 0, 0, 0, 0, 1, 0, 0, 1}

Fig. 8 Adjacency matrix as a basic feature vector of the zone relationships.

4.1.3. Export and run controls

The interface (Fig. 7) has buttons for executing calculations ("Run") and exporting data in various formats, enabling

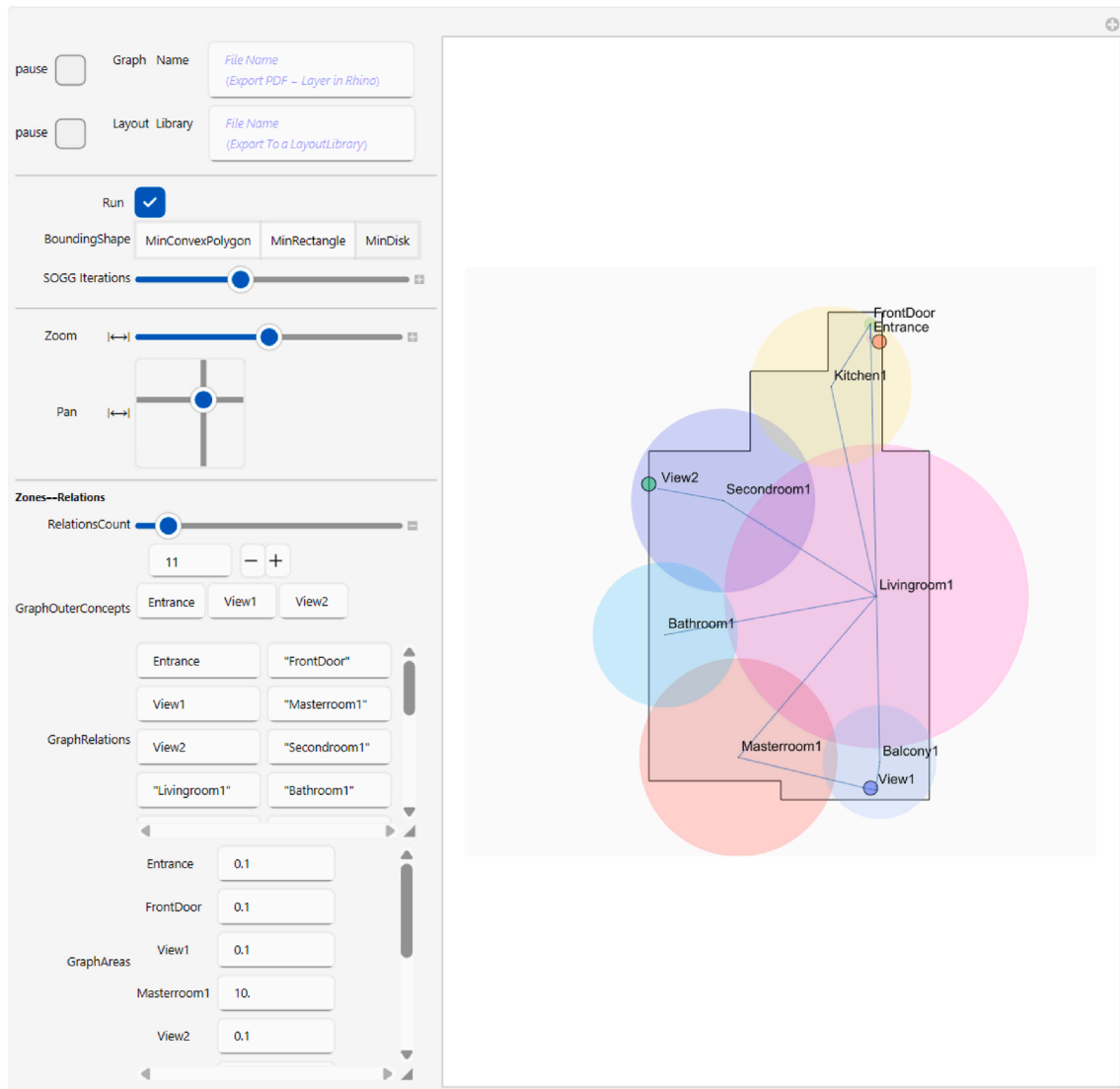


Fig. 7 Interface of the SOAG-based floor diagram tool with randomly coloured circular zones.

Entrance	{3., 1.5, -3., -3., -3., -3., -3., -3., -3., -3.}
FrontDoor	{1.5, 3., -3., -3., -3., -3., 1.5, -3., -3., 1.5}
View1	{-3., -3., 3., 1.5, -3., -3., -3., -3., 1.5, -3.}
Masterroom1	{-3., -3., 1.5, 3., -3., -3., 1.5, -3., -3., -3.}
View2	{-3., -3., -3., -3., 3., 1.5, -3., -3., -3., -3.}
Secondroom1	{-3., -3., -3., -3., 1.5, 3., 1.5, -3., -3., -3.}
Livingroom1	{-3., 1.5, -3., 1.5, -3., 1.5, 3., 1.5, 1.5, 1.5}
Bathroom1	{-3., -3., -3., -3., -3., -3., 1.5, 3., -3., -3.}
Balcony1	{-3., -3., 1.5, -3., -3., -3., 1.5, -3., 3., -3.}
Kitchen1	{-3., 1.5, -3., -3., -3., -3., 1.5, -3., -3., 3.}

Fig. 9 Feature vector after replacing values, as described in Steps 4–5 of the algorithm shown in Fig. 6.

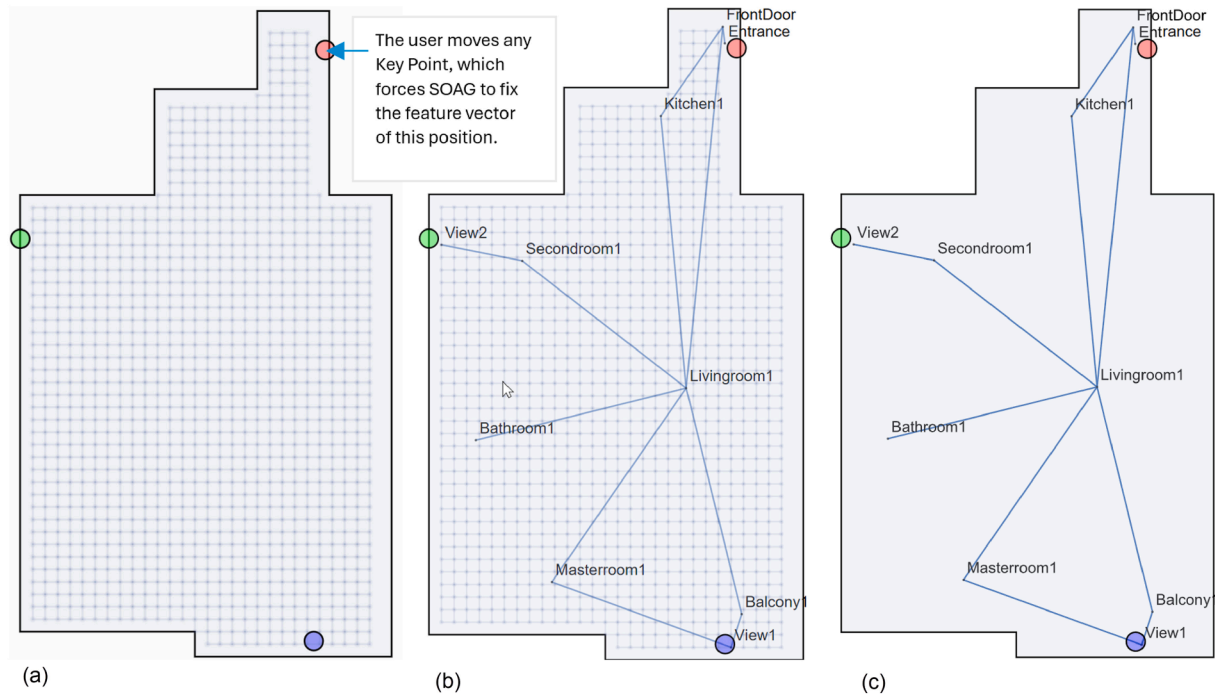


Fig. 10 Process from the stage of localising key points (a) to the best-matching unit after training the SOAG (b), through to connecting the edges based on the relationship requirements (c).

integration with external tools, such as Rhino or layout libraries.

Based on the inputs required for adjacency, the relationships in the input fields of the experiment in Fig. 7 are as follows:

{Frontdoor $\Leftrightarrow$ Livingroom, View1 $\Leftrightarrow$ Masterroom1, View2 $\Leftrightarrow$ Secondroom1, View1 $\Leftrightarrow$ Reception, Frontdoor $\Leftrightarrow$ Kitchen, Entrance $\Leftrightarrow$ Frontdoor, Living $\Leftrightarrow$ Bathroom1}.

5. Results and discussion

The SOAG can generate floor plan diagrams in real time based on user inputs, such as topological, geometrical and contextual constraints. The SOAG algorithm integrates user-defined key points, such as entrance position and outer viewpoints, into the final layout, successfully localising zones and differentiating between related and unrelated zones by initialising the feature vector with the adjacency matrix values. It then

transforms the matrices by substituting values that represent each element in the vector with a unique high value, thereby assisting users in performing adjacency clustering, as shown in Step 4 in Fig. 6.

Introducing a new distance function expands the capabilities of the algorithm beyond the generation of standard two-dimensional SOM map layouts with a Euclidean metric. A graph grid is more effective than a metric grid because it can represent functional movement rather than physical distance. Consequently, if two points are close in a metric grid and there is an obstacle, such as a wall, between them, the metric grid cannot recognise this feature, whereas a graph grid can effectively represent obstacles, such as walls, by removing connections between the two points. A graph grid can also define exclusion zones, ensuring that spaces are not generated in restricted areas. This adaptability helps to satisfy the requirements of both spatial continuity and zoning constraints.

Replacing the binary values of the adjacency relations (Fig. 6, Step 4) improves the ability of the algorithm to distinguish differences in the vector. The use of a cosine distance function (Fig. 6, Step 14), instead of the standard Euclidean distance, to evaluate the similarity of vectors allows a focus on the directionality of relationships rather than their magnitude. This is ideal for evaluating graphical data with asymmetric connections, where the feature vector structure is more important than the strength of its relationships. In such cases, the relationships between zone features, and their adjacency, are more important than their absolute positions. The SOAG training process preserves topological relationships, which are encapsulated in the neurons' updated weights of SOAG (Fig. 11), leading to layouts aligned with practical needs and supporting real-time decisions in the initial design stages. In other words, SOAG simultaneously deals with both the latent space and geometrical metric space used by architects.

The interactive interface allows users to modify key parameters, such as adjacency relationships, space areas and the locations of key points. It also achieves real-time performance (less than 0.2 s per alternative) using a laptop with a CPU-only configuration, demonstrating the efficiency of the system even without GPU acceleration. Consequently, it enables the exploration of design alternatives while reducing the iterative workload typically associated with initial floor plan generation. The graphical output, including the grid representation and adjacency

connections, provides an intuitive visualisation of the spatial relationships. Using graph nodes and edges effectively communicates the spatial configuration, facilitating ease of interpretation by architects and designers. The matrix of maximum weight distances per cluster, known as the U-matrix (Kohonen, 2001), shows the boundaries (as dark lines) between the clusters in Fig. 12. The current implementation of the zones' areas uses an MWVD (Dong, 2008) with a weighted distance function, where the weight is determined as a ratio to the areas of the zones as follows:

$$d_v(p_v, p_i) = \frac{\sqrt{(x_v - x_i)^2 + (y_v - y_i)^2}}{\text{area}_i}, \quad (1)$$

where d_v is the distance in each Voronoi region that represents the region of a zone, p_v is the zone graph vertex coordinate and p_i is a point within the polyline. The distance is relative to the area of each zone. Following SOAG training, the impacts of the areas are replaced with ratios, which can be seen as a limitation when areas are much larger than the adjacent elements. Key points, such as viewpoints, may have a small area or weight because they do not act as real zones, but rather as attractor points to adjacent zones (Figs. 7 and 10). Moving the key points generates more alternatives in real time without the need to have similar examples in the data for their localisation, as shown in Fig. 13. Furthermore, it is necessary to refer to

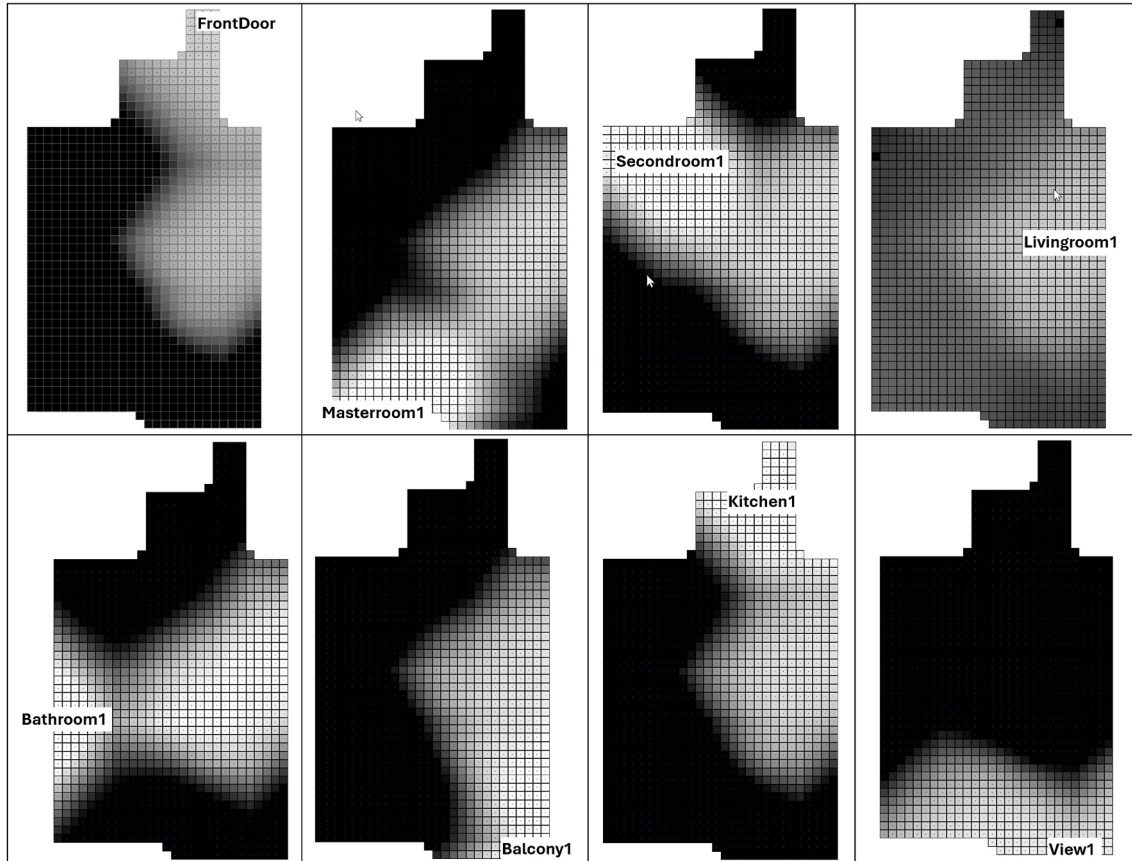


Fig. 11 Neuron weights for some zones. White cells = 100% likelihood for the zone; black cells = 0% likelihood for the zone.

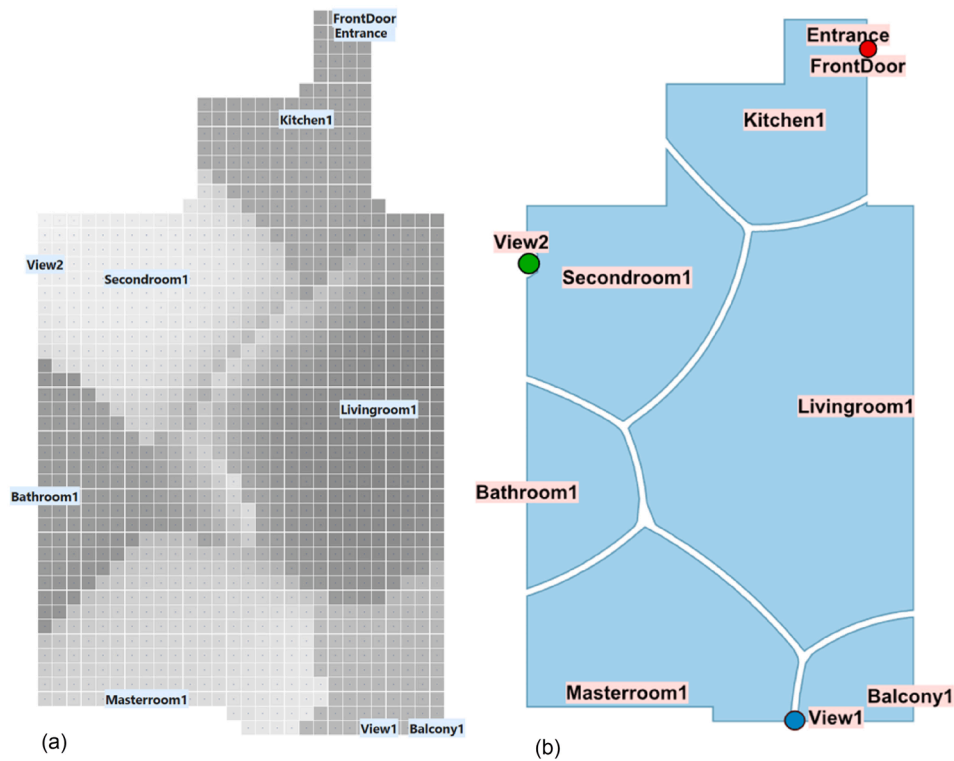


Fig. 12 (a) U-matrix showing dark borders between the final cluster after training SOAG, (b) the final multiplicative weight Voronoi diagram (MWVD) showing the best matching units (BMUs) for each zone.

the essence of MWVD, which was integrated inside the learning process of SOAG in different mathematical formats, in Step 15 of Fig. 6. Updates to the neurons' weights are influenced by multiplying the graph distances by the square root of the areas according to the BMU selection.

This paper does not aim to automate the entire process of floor plan generation, but rather to focus on creating a likelihood-based canvas for placing initial zones for designers, allowing them to modify initial localisations before navigating a probabilistic setup of a bubble diagram. The generalisability of the proposed approach can be considered at a theoretical level. Within the scope of this study, further experiments and refinements are required to strengthen its applicability. To provide an initial indication of this potential, we also tested the algorithm on additional regular and irregular shapes, the results of which are presented in Appendix A.

5.1. Qualitative comparison

SOAG exhibits an ability to generate probabilistic regions of zones that overlap the same GT in Fig. 13. Unlike most dominant research approaches, the proposed approach does not rely on a training process; instead, we used only the relations and areas as a GT for comparison. Thus, from a qualitative comparison, similar results are obtained; additionally, changing the positions of key points (Fig. 13, right column) leads to new probabilistic-based bubble zone locations.

5.2. Quantitative comparison

The quantile plot (Fig. 14) compares the ground truth (GT) areas, represented by the diagonal dotted line, with the areas generated by SOAG, shown as scattered points. Each data point corresponds to an area measurement in mm^2 . When the points are closer to the diagonal, a strong correlation between the GT values and the generated values is indicated, whereas deviations from the line highlight differences. The clustering of most points along the diagonal suggests that our method achieves a high level of accuracy for smaller and moderate areas; however, some noticeable deviations appear above 8000 mm^2 , reflecting slight over- or under-estimations by our method. Overall, the visualisation demonstrates that the approach aligns well with the GT, with only a few outliers at higher ranges. Although these results suggest overall good agreement, additional experiments are needed to obtain a more comprehensive assessment of the method's performance. The Hausdorff metric, also known as the Hausdorff distance (HD), was used to measure the difference between the two regions, namely the GT zones and the zones generated by the SOAG. HD is utilised in numerous computer vision applications and studies, particularly for shape matching (Alt et al., 2003; Alt and Guibas, 2009). As the areas were in mm^2 , the histogram shows some differences in shapes (Fig. 15); even here, a similarity in the likelihood positions of zones and areas, rather than in shape, is needed.

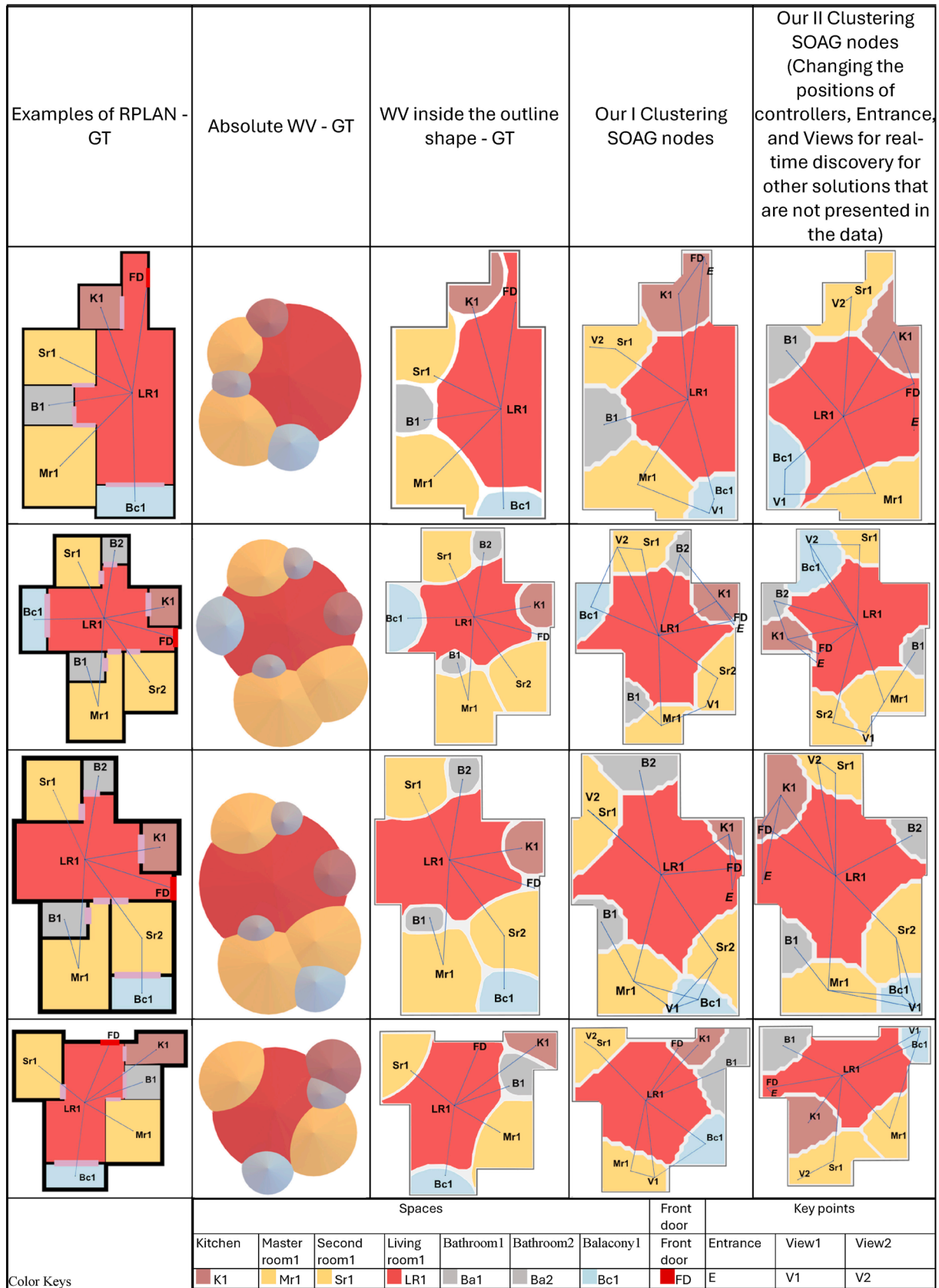


Fig. 13 Multiple diagram alternatives with key points moved to different locations.

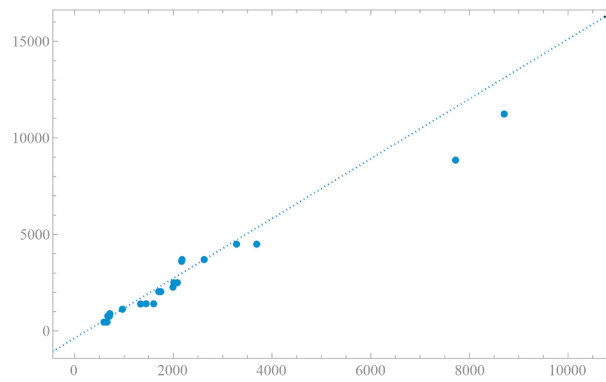


Fig. 14 Quantile plot comparing the diagonal line representing the ground truth (GT) areas; points denote the areas that were generated by the SOAG.

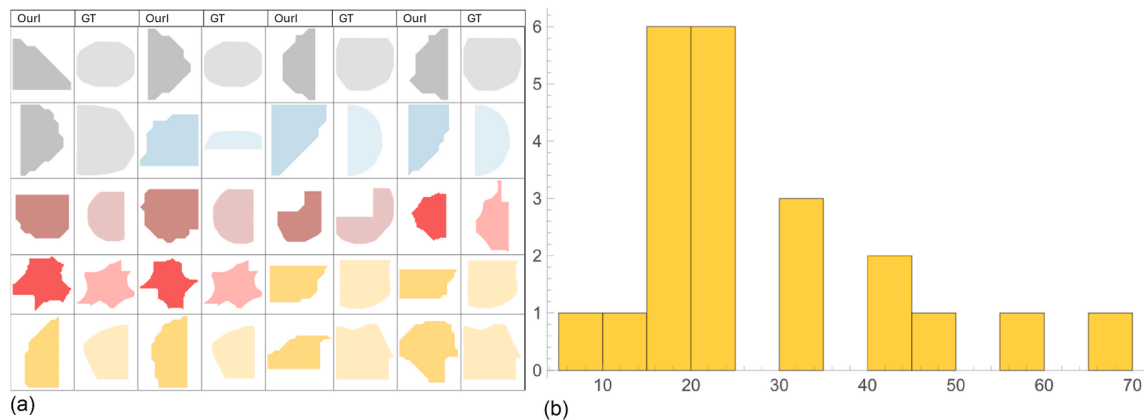


Fig. 15 (a) Comparison of the WV of zones and the SOAG generated, (b) histogram for the Hausdorff distance.

6. Conclusions

The proposed SOAG method generates floor plan diagrams while creating initial outlines of convex or concave boundaries. It refines the traditional SOM techniques to suit architectural design applications and facilitates real-time human inputs; as such, it is a valuable for architects seeking to balance efficiency and creativity. It integrates user input and interaction at various stages in the process, allowing users to define adjacency constraints and set the positions of key points while ensuring that outcomes are adaptable to project needs without being driven by pre-trained floor plan data. Although we evaluated the developed approach using residential layouts, the method is adaptable to different kinds of floor plan typologies. This study demonstrates the potential of training a machine learning algorithm to solve problems on the fly, without exposure to previous solutions, in support of the early stages of architectural design by automating and optimising the generation of floor plan diagrams. The principal contribution of this study is in accelerating the automation of the initial planning and design stages. Further studies are needed to expand beyond the limitations of the proposed method and develop its application to more complex functional requirements.

7. Limitations and future work

The primary limitation of this method lies in the accuracy of the area distribution, which relies only on the MWVD and involves area ratios rather than absolute values. Efforts should be made to expand the algorithm's ability to include additional design constraints, such as generating multiple floor plan diagrams. Additionally, integrating the method with established design tools and platforms, such as Rhino3D, could further streamline workflows and enhance its utility in professional practice. Moreover, validating the approach using more extensive and diverse floor plan scenarios, including complex layouts and multi-floor building designs, would help to demonstrate its robustness and versatility in varied architectural contexts. Finally, to avoid occasional intersection of graph edges when the key points are adjusted, the key points may be slightly repositioned through real-time interactions.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors would like to thank Ajman University for their APC support.

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.foar.2025.11.011>.

References

- Akinduko, A.A., Mirkes, E.M., Gorban, A.N., 2016. SOM: stochastic initialization versus principal components. *Inf. Sci.* 364–365, 213–221.
- Alexander, C., 1964. *Notes on the Synthesis of Form*. Harvard University Press, Cambridge, MA.
- Alt, H., Braß, P., Godau, M., Knauer, C., Wenk, C., 2003. Computing the Hausdorff Distance of Geometric Patterns and Shapes, pp. 65–76.
- Alt, H., Guibas, L.J., 2009. Discrete geometric shapes: matching, interpolation, and approximation A survey. In: *Alt2009DiscreteGS*.
- Bonabeau, E., Hénaux, F., 1998. Self-organizing maps for drawing large graphs. *Inf. Process. Lett.* 67, 177–184.
- Dong, P., 2008. Generating and updating multiplicatively weighted Voronoi diagrams for point, line and polygon features in GIS. *Comput. Geosci.* 34, 411–421.
- Egor, G., Sven, S., Martin, D., Reinhard, K., 2020. Computer-aided approach to public buildings floor plan generation. *Magnetizing floor plan generator*. In: *Procedia Manufacturing*. Elsevier B.V., pp. 132–139.
- Emmons, P., 1998. The cosmogony of bubble diagrams. In: Barton, C. (Ed.), *Constructing Identity: Annual Meeting Proceedings of the 86th ACSA*. ACSA, pp. 420–425.
- Fortin, G., 1978. BUBBLE: relationship diagrams using iterative vector approximation. In: *Proceedings of the 15th Design Automation Conference, DAC'78*. IEEE Press, Las Vegas, pp. 145–151.
- Franz, G., Mallot, H.A., Wiener, J.M., Neurowissenschaft, K., 2005. Graph-based models of space in architecture and cognitive science: a comparative analysis. In: *Architecture, Engineering and Construction of Build Environments*. 17th International Conference on Systems Research, Informatics and Cybernetics (INTERSYMP 2005). International Institute for Advanced Studies in Systems Research and Cybernetics, pp. 30–38.
- Galle, P., 1981. An algorithm for. Exhaustive generation of building floor plans. *Commun. ACM* 24, 813–825.
- Guo, Z., Li, B., 2017. Evolutionary approach for spatial architecture layout design enhanced by an agent-based topology finding system. *Front. Archit. Res.* 6, 53–62.
- Hsu, Y.-C., Krawczyk, R.J., 2003. New generation of computer aided design in space planning Methods-a survey and A proposal. In: *Proceedings of the 8th International Conference on Computer Aided Architectural Design Research in Asia*. CAADRIA 2003. Bangkok, pp. 101–116.
- Hu, R., Huang, Z., Tang, Y., Van Kaick, O., Zhang, H., Huang, H., 2020. Graph2Plan: learning floorplan generation from layout graphs. *ACM Trans. Graph.* 39 (4), 118.
- Kakooee, R., Dillenburger, B., 2025. Enhancing architectural space layout design by pretraining deep reinforcement learning agents. *J. Comput. Des. Eng.* 12, 149–166.
- Kakooee, R., Dillenburger, B., Kakooee, R., 2024. ETH library reimagining space layout design through deep reinforcement learning rights/license: creative commons attribution 4.0 international reimagining space layout design through deep reinforcement learning. *J. Comput. Des. Eng.* 11, 43–55.
- Kohonen, T., 2001. *Self-Organizing Maps*, Springer Series in Information Sciences. Springer, Berlin Heidelberg, Berlin, Heidelberg.
- Lueder, C., 2012. Diagram ecologies. In: Cox, P., Plimmer, B., Rodgers, P. (Eds.), *Diagrammatic Representation and Inference*. Springer, Canterbury, pp. 214–231.
- Meyer, B., 1998. Self-organizing graphs — a neural network perspective of graph layout. *Lect. Notes Comput. Sci.* 1547, 246–262.
- Mitchell, W.J., Dillon, R., 1972. A Polyomino assembly procedure for architectural floor planning. In: Mitchell, W.J. (Ed.), *Proceedings of Environmental Design Research Association and AIA-Architect-Researchers*. American Inst. of Architects, Los Angeles.
- Nassar, K., 2019. New advances in the automated architectural space plan layout problem. In: *EG-ICE 2010 - 17th International Workshop on Intelligent Computing in Engineering (Nottingham)*.
- Nauata, N., Hosseini, S., Chang, K.-H., Chu, H., Cheng, C.-Y., Furukawa, Y., 2021. House-GAN++: generative adversarial layout refinement network towards intelligent computational agent for professional architects. In: *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13632–13641.
- Nobbs, P.E., 1937.. In: *Design a Treatise on the Discovery of Form*. Oxford University, London.
- Ruch, J., 1978. Interactive space layout: a graph theoretical approach. In: *Proceedings of the 15th Design Automation Conference, DAC'78*. IEEE Press, pp. 152–157.
- Shaviv, E., 1985. Layout design problems: systematic approaches. In: *CAAD Futures Digital Proceedings*. Delft.
- Su, M.-C., Liu, T.-K., Chang, H.-T., 1999. An efficient initialization scheme for the self-organizing feature map algorithm. In: *IJCNN'99. International Joint Conference on Neural Networks. Proceedings (Cat. No.99CH36339)*, pp. 1906–1910. Washington, DC, USA, 1999.
- Veloso, P., Rhee, J., Bidgoli, A., Ladron, M., Guevara, D.E., 2022. BUBBLE2FLOOR A pedagogical experience with deep learning for floor plan generation. In: Ameijde, J. van, Gardner, N., Hyun, K.H., Luo, D., Sheth, U. (Eds.), *Proceedings of the 27th CAADRIA Conference*, pp. 373–382. Sydney.
- Veloso, P., Rhee, J., Krishnamurti, R., 2014. Multi-agent space planning A literature review (2008-2017). In: *XVIII Conference of the Iberoamerican Society of Digital Graphics - SIGraDi: Design in Freedom*. SIGraDi, Montevideo, pp. 115–119.
- Wang, X.Y., Yang, Y., Zhang, K., 2018. Customization and generation of floor plans based on graph transformations. *Autom. Construct.* 94, 405–416.
- White, E.T., 1975. *Concept Sourcebook: a Vocabulary of Architectural Forms*.
- Wu, W., Fu, X.M., Tang, R., Wang, Y., Qi, Y.H., Liu, L., 2019. Data-driven interior plan generation for residential buildings. *ACM Trans. Graph.* 38 (6), 234.
- Yildiz, B., Cagdas, G., Zincir, I., 2023. Deep Architectural Floor Plan Generation: an Approach for Open-Planned Residential Spaces.
- Zaghloul, M., 2015. Machine-learning aided architectural design self-organizing map generates in-between design alternatives chair for computer aided architectural design. In: *GA2015—XVIII Generative Art Conference*, pp. 283–293.
- Zhang, H., Savov, A., Dillenburger, B., 2024. MaskPLAN: masked generative layout planning from partial input. In: *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 8964–8973.
- Zhang, Y., Pang, A., 2016. Graph layout with versatile boundary constraints. *J. Graph Algorithm Appl.* 20, 435–459.